DPO 论文问答记录

Direct Preference Optimization: Your Language Model is Secretly a Reward Model（arXiv:2305.18290v3，NeurIPS 2023）

按论文章节组织，章节内按论文顺序；最后一节是论文之外的延伸问题。

---

3 Preliminaries：DPO 之前的 RLHF

Q：第 3 节里的式 (2) 和式 (3) 是什么?

A：这两个式子都不是 DPO 的东西。第 3 节在复述「DPO 出现之前的标准做法」：式 (2) 是第二阶段训练 reward model 的损失，式 (3) 是第三阶段用 RL 调 policy 的目标。它们正是 DPO 要取代的对象。

符号先钉死：$\mathcal{D}$ 是偏好数据集，每条是一个三元组 (x, y_w, y_l)，x 是 prompt，y_w、y_l 是同一个 prompt 下的两个 response，人类认为 y_w 更好（w = win，l = lose）。r_ϕ 是 reward model，参数为 ϕ；σ 是 sigmoid，把任意实数压进 $(0,1)$；$\mathbb{E}$ 落到代码里就是对 batch 求平均。

$$\mathcal{L}_R(r_\phi, \mathcal{D}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right] \tag{2}$$

式 (2) 从里往外读：先取两条 response 的分差，过 σ 变成「人类会偏好 y_w 的概率」，取负 log 就是二分类交叉熵，最后在整个数据集上平均。一句话：训一个网络，让它给人类偏好的那条打更高的分。它只约束分差、不约束绝对值——所有分数统一加 100，loss 一个字都不变。

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(y|x)}\left[r_\phi(x, y)\right] - \beta\, \mathbb{D}_{\mathrm{KL}}\left[\pi_\theta(y \mid x) \,\|\, \pi_{\mathrm{ref}}(y \mid x)\right] \tag{3}$$

式 (3) 是目标函数，要往大了推。第一项：让当前的 π_θ 现场生成 y，交给 r_ϕ 打分再求平均。第二项：π_θ 偏离 π_{ref}（通常是冻结的 SFT 模型）越远，扣分越多，β 控制扣多狠。整体是一个拉锯：往高分方向跑，但别跑离原模型太远。论文给 KL 约束的三条理由：r_ϕ 只在它的训练分布附近才准；维持生成多样性；防止 mode collapse。

Q：为什么式 (3) 必须用 RL 来解?

A：因为第一项里的 y 是从 π_θ 采样出来的离散 token 序列，采样这个动作不可导，梯度传不回 θ，只能上 PPO 这类 RL 算法。对照式 (2)：那里的 y_w、y_l 是数据集里固定的字符串，所以能直接求导。DPO 要做的，就是在完全不改式 (3) 的前提下，把它变回式 (2) 那种能直接梯度下降的形状。

---

4 Direct Preference Optimization

Q：第 4 节的推导（式 4–7）在干什么？是不是 RM + PPO 能通过数学推导换成一个等价的单步过程，这个过程就是 DPO?

A：基本是。更准确的说法是：式 (3) 这个目标原本只能靠 RM + PPO 两步去逼近，推导发现它可以直接改写成一个关于 policy 的损失，用普通梯度下降一步解掉。RM 并没有被绕开，而是被吸收进了 policy——这就是标题里 “Your Language Model is Secretly a Reward Model” 的意思。

推导本身不用会，只需拿住三件事：

1. **式 (3) 的最优解有闭式**（式 4）。从 π_{ref} 出发，给每条 response 乘上权重 $\exp(r/\beta)$：奖励高的被放大，奖励低的被压下去。唯一挡路的是归一化常数 $Z(x)$——它要对所有可能的 response 求和，算不出来。
2. **奖励和 policy 可以互相翻译**（式 5）。把式 4 反过来写，奖励就能表示成 $\beta \log \frac{\pi}{\pi_{\text{ref}}}$ 再加一个 $\beta \log Z(x)$。
3. **$Z(x)$ 会被抵消**。它只依赖 x，而 Bradley-Terry 只看同一个 prompt 下两条 response 的奖励之差，两个 $Z(x)$ 一减就没了。

把翻译后的结果代回式 (2)，就得到式 (7)：形状和式 (2) 一模一样，只是 $r_\phi(x, y)$ 被整个换成了 $\beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$。

Q：DPO 是从 PPO 推出来的吗?

A：不是。DPO 是从式 (3)，也就是 RLHF 的目标推出来的。PPO 是求解式 (3) 的一种算法，DPO 是另一种；两者共享同一个目标，谈不上谁从谁推出来。

Q：那 PPO 能做的事，DPO 都能做吗?

A：只在「从一份固定的成对偏好数据学习」这个场景里成立。PPO 训练时会现场采样，所以它能用没有标注的 prompt，也能接任何能打分的奖励函数；DPO 只能吃离线的成对数据。论文第 7 节自己把「用不上无标注 prompt」列为局限。

Q：为什么 DPO 的数据必须成对，而且两条 response 必须来自同一个 prompt?

A：这是推导本身的硬性要求，不是工程习惯：那个算不出来的 $Z(x)$，只有在同一个 prompt 下两项相减才会抵消。所以 DPO 没法用单条样本训练。

Q：跳过所有推导，DPO 最终推出了什么?

A：只有一个损失函数：

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

读法：看 policy 相对 reference 在 y_w 上把概率抬高了多少、在 y_l 上抬高了多少，前者减后者，乘 β，过 sigmoid 再取负 log。$\beta \log(\pi_\theta/\pi_{\text{ref}})$ 这个量论文称为 implicit reward。

推导证明了四件事，直接当结论用：

- 这个损失的最优解就是 RLHF（RM + PPO + KL 约束）的最优解，所以训练时不需要 RM、不需要 PPO、不需要采样，只是在固定的偏好数据集上做普通梯度下降。
- 训完之后 $\beta \log(\pi_\theta/\pi_{\text{ref}})$ 本身就是一个合法的 reward。附录 B 代码里的 `rewards = beta * (pi_logps - ref_logps)` 算的就是它，可以用来监控训练。
- 这种写法不损失表达能力：RM 能表示的偏好，DPO 都能表示（Theorem 1）。
- 梯度把 y_w 的概率往上抬、把 y_l 往下压，并按「implicit reward 当前把这一对排错了多少」加权。这个权重不能去掉，去掉就成了 Unlikelihood，模型会退化成反复输出 `when when when...`（附录表 3）。

5-6 DPO 与 PPO：效果与原因

Q：DPO 和 PPO 的效果是一样的吗？效率变高只是因为不用单独训练 RM？

A：两句都要修正。

先说效果：DPO 更好，不是一样。

- 情感任务（图 2 左）：DPO 的 reward/KL 折中**严格支配** PPO，同样的 KL 预算下，DPO 每一点的奖励都更高。
- 摘要（图 2 右）：DPO 在温度 0 时 win rate 约 61%，PPO 在它自己的最优温度 0 下为 57%；人类评测中，DPO（温度 0.25）的样本有 58% 被判为优于 PPO（温度 0）。
- 对话：作者试了公开的 PPO RLHF 模型，找不到任何 prompt 或温度能让它超过没微调过的 base Pythia-2.8B。

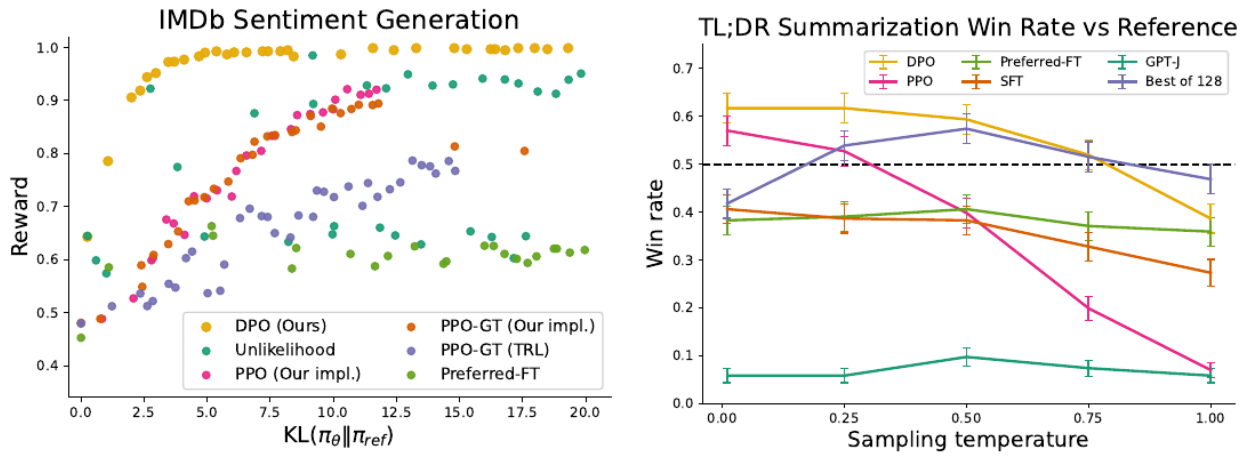


Figure 2: **Left.** The frontier of expected reward vs KL to the reference policy. DPO provides the highest expected reward for all KL values, demonstrating the quality of the optimization. **Right.** TL;DR summarization win rates vs. human-written summaries, using GPT-4 as evaluator. DPO exceeds PPO's best-case performance on summarization, while being more robust to changes in the sampling temperature.

Figure 2：左为 IMDb 情感任务的 reward–KL frontier，右为 TL;DR 摘要在不同采样温度下的 win rate

最硬的一条证据：DPO 连 **PPO-GT** 都赢了。PPO-GT 直接拿真实奖励函数（情感分类器）去优化，完全没有 RM 拟合误差，结果仍然输给 DPO。这说明差距不在「RM 训得准不准」，而在 RL 优化这一步本身。

再说效率：省掉的不止 RM，而是三样——RM 的那轮训练、**训练过程中的采样**（最贵的一项，自回归逐 token 生成）、value network。保留下来的 π_{ref} 冻结只做前向，所以不是「4 个模型变 1 个」，而是变成 2 个、其中 1 个不训。

「效率」在这篇论文里其实指两件事：一是工程开销（模型少、不采样、跑得快）；二是优化效率（同样的 KL 预算下能拿到多少奖励）。图 2 左说的是后者，与省了几个模型无关。

边界：论文自己的措辞是 “as well as or better than”，不是全面碾压。三个任务的证据强度依次是情感 > 摘要 > 对话；对话那组的 PPO 对照用的是外部公开 checkpoint，偏弱。

Q：为什么 DPO 的效果更好?

A：论文给出的正面机理解释只有 5.2 节一条，其余都是实验观察。

5.2 节的论证：如果按 actor-critic 的方式把式 (3) 整理成可优化的形式（式 10），目标里会冒出一个归一化项，论文把它解读为 π_{ref} 的 soft value function。这一项不影响最优解，但如果估不准，policy gradient 的方差会很高，训练就不稳。PPO 的两种应对都不干净：要么学一个 value network 去逼近它（这个网络本身就难训），要么用人类 completion 当 baseline 做单样本估计（噪声很大）。而 DPO 的重参数化给出的奖励**根本不需要任何 baseline**。方差高之所以会体现为效果差，我的理解是：梯度方向每步都在抖，只能小步走，同样的更新次数爬不了那么高；抖动还会把 policy 推离 π_{ref}，KL 预算被消耗了，奖励却没换回来——落到图 2 左上，就是 frontier 更低。这一步论文没有展开。

PPO-GT 是排除法证据：它排除了「RM 不准」这个解释，把矛头指向 RL 优化本身，正好和 5.2 节的诊断对上。（早先说过的「PPO 有 RM 误差和 RL 误差两处叠加」不准确，PPO-GT 恰好否掉了 RM 误差那一半。）

另一条论文反复提到、但没上升为机理的：DPO 是在固定的离线数据集上做最大似然，梯度是解析的；PPO 的梯度要靠现场采样来估计。这也解释了 DPO 为什么对采样温度那么鲁棒——DPO 训练时根本没有采样环节，温度只在评测时出现。

注意归因：3.5 节梯度里的动态权重，解释的是「DPO 为什么不退化」（对照 Unlikelihood），**不是**「DPO 为什么比 PPO 好」。

可信度：5.2 节是一个诊断，不是定理。论文没做最能定论的对照实验——给 PPO 补上一个足够准的归一化项，看它能否追平。情感任务的结论最硬（受控、真实奖励已知、22 次 run 扫出完整 frontier，外加 PPO-GT 排除法）；对话任务基本回答不了「为什么」，只能说那组 PPO 没跑起来。另外论文强调几乎没调 DPO 的 β，但 TL;DR 用的是 0.5 而非默认的 0.1，说明 β 至少动过一次；也没有证据表明 PPO 被调到了最好状态。

---

6 实验：训练规模与复现

Q：论文原文的训练规模是怎样的?

A：原文写明的：

任务	模型	偏好数据
IMDb 情感	GPT-2-large（约 7.7 亿参数）	自己造：25000 个前缀 × 4 条 completion，每个前缀组 6 对，共 15 万对
TL;DR 摘要	GPT-J 6B（CarperAI 开源的 SFT 模型）	Stiennon et al. 的人类偏好数据（论文没写用了多少条）
Anthropic-HH 对话	Pythia-2.8B（先在 chosen 上做 Preferred-FT 得到 π_{ref}）	170k 条对话

三个任务共用超参：$\beta = 0.1$（TL;DR 用 0.5）、batch size 64、RMSprop、学习率 1e-6、前 150 步线性 warmup。论文没提 LoRA，1e-6 这个学习率也是全参微调的量级。

论文没写总步数，只能从图 3 右的横轴看出对话任务训到约 3300 步，按 batch 64 约 21 万对样本。

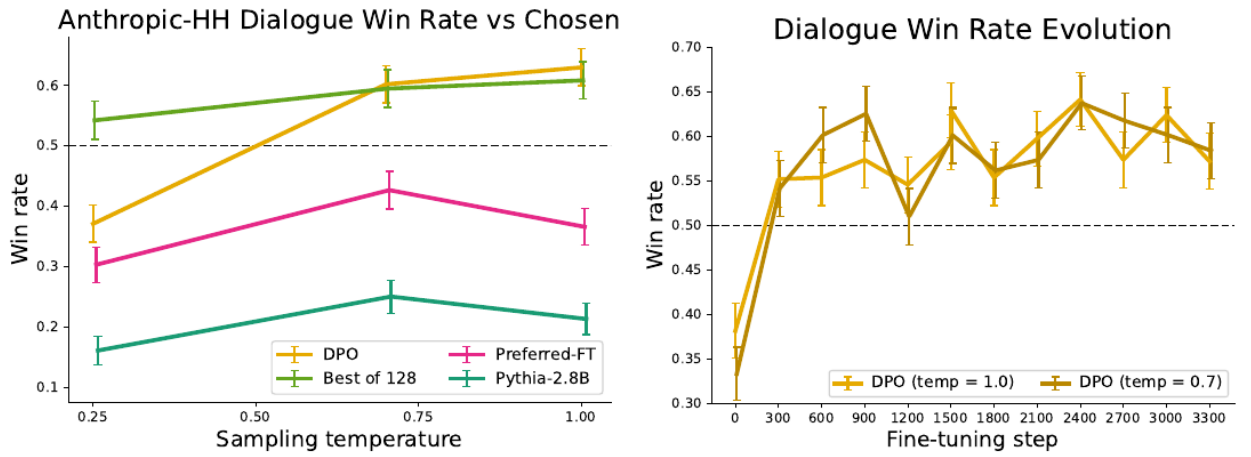


Figure 3: **Left.** Win rates computed by GPT-4 for Anthropic-HH one-step dialogue; DPO is the only method that improves over chosen summaries in the Anthropic-HH test set. **Right.** Win rates for different sampling temperatures over the course of training. DPO's improvement over the dataset labels is fairly stable over the course of training for different sampling temperatures.

Figure 3：左为 Anthropic-HH 对话 win rate，右为训练过程中的 win rate 演化

论文自身的不一致：第 6 节说 IMDb 的 SFT「训到收敛」，附录 C.1 又说「在 IMDB 的一个子集上训 1 个 epoch」。

Q：原文在什么显卡上跑的？要跑多久？

A：论文正文和附录都没写——GPU 型号、卡数、训练时长一个都没出现，致谢只说部分算力来自 Stanford CRFM。

作者的官方代码仓库 README 写了：**4 张 80GB A100**。以对话任务为例，Pythia-2.8B 在 Anthropic-HH 上 SFT 约 **1 小时 30 分**，DPO 约 **2 小时 45 分**，使用 PyTorch FSDP 把模型切到多卡上，开 bf16 混合精度（README 称约快 50%）。命令是现成的：

```
# SFT
python -u train.py model=pythia28 datasets=[hh] loss=sft \
    exp_name=anthropic_dpo_pythia28 \
    gradient_accumulation_steps=2 batch_size=64 eval_batch_size=32 \
    trainer=FSDPTrainer sample_during_eval=false \
    model.fsdp_policy_mp=bfloat16

# DPO（接在 SFT 后面，beta=0.1）
python -u train.py model=pythia28 datasets=[hh] loss=dpo loss.beta=0.1 \
    exp_name=anthropic_dpo_pythia28 \
    gradient_accumulation_steps=2 batch_size=64 eval_batch_size=32 \
    trainer=FSDPTrainer sample_during_eval=false \
    model.fsdp_policy_mp=bfloat16 \
    model.archive=/path/to/archive/from/sft/LATEST/policy.pt
```

两个限制：这是代码仓库的示例，不是论文里的实验记录，不能保证设置与论文完全一致；README 只给了对话任务这一组，IMDb 和 TL;DR 的硬件与时长都没写。

Q：在自己的 RTX 3090（24GB）上能复现吗？要跑多久？

A：以下是按参数量和序列长度估的量级，不是论文数据，实际可能差一两倍：

任务	能不能跑	估计时长
IMDb + GPT-2-large	能，可全参微调（权重、梯度、优化器状态加冻结的 π_{ref} 合计 11GB 左右）	SFT 约半小时；生成并打分造 15 万对约 0.5–1 小时；DPO 跑完 15 万对一个 epoch 约 1–2 小时
HH + Pythia-2.8B	全参放不下，要用 LoRA	按论文的 3300 步，约两天
TL;DR + GPT-J 6B	只能用 QLoRA	按「天」计

这个估算和 README 对得上：4 张 A100 的 bf16 算力约是一张 3090 的十几倍，2 小时 45 分乘上去差不多两天。后两个任务的评测还要用 GPT-4 判 win rate，需要 API 费用；IMDb 用情感分类器打分，不花钱。

建议只在 3090 上复现 IMDb：它是唯一能原样跑完的任务；它有 ground-truth reward，可以自己画出图 2 左的 reward–KL frontier；序列短、迭代快。先抽 2 万对的子集，每个 β 约 15–20 分钟，扫完论文用的 0.05 / 0.1 / 1 / 5 后再上全量，整套半天到一天。之后算力充裕时，直接用官方仓库复现 HH 那组——它是唯一有官方命令和官方时长可对照的设置，4 张 A100 半天出结果，可以用来校验环境。GPT-J 6B 全参 DPO 单张 80GB 放不下 policy、π_{ref} 和优化器状态，需要 FSDP 跨多卡，耗时会是 Pythia-2.8B 那组的好几倍（估计）。

附：学习路线与延伸（论文之外）

Q：DPO 太偏数学了，还有必要学吗？现在 DPO 的应用场景多吗？

A：要学，而且 VLM 细粒度后训练这个方向尤其要学；但推导恰恰是最不需要学的那部分。在后训练算法里，DPO 其实是数学最轻的一档：损失落到代码就是附录 B 那十几行，没有 GAE、没有 clip、没有 value network、没有 rollout buffer。觉得它偏数学，是因为论文花了大量篇幅证明这个损失是合法的，而用它的人不需要重走这条证明。

应用现状（2026 年 10 月检索）分两边看：

- **多模态方向**：2026 年 5–9 月的多模态后训练论文里，大量仍是 DPO 变体，且高度集中在幻觉抑制和细粒度视觉 grounding 上，例如 PEA-DPO、P²-DPO、Context-Calibrated PO（Context Blindness in DPO）、Groc-PO、ADAPT、MCPO、Graded Preference Optimization。在这个方向，DPO 是默认 baseline，也是审稿人的默认参照系。

- **纯文本方向**：reasoning 已被 RLVR / GRPO 接管——数学和代码能写 verifier，有了可验证奖励就不必走偏好数据。但 DPO 没有退场，而是迁移到写不出 verifier 的那一半问题上：风格、安全、价值观、幻觉。2026 年 9 月仍有工作用 iterative DPO 研究 reward hacking 引发的 misalignment，也有把 GRPO、SFT、DPO 并列审计的工作；还有工作直接从 DPO 的角度重新理解 GRPO（“It Takes Two: Your GRPO Is Secretly DPO”）。

VLM 的幻觉之所以留在 DPO 这边，我的判断是：「图里有没有这个物体、描述得够不够细」很难写成一个可靠的自动 verifier，只能退回成对偏好。

Q：跳过推导之后，还需要学什么？

A：

- **β 的含义**：KL 约束的强度。β 大，紧贴 π_{ref}，保守；β 小，追偏好更激进，也更容易跑偏。论文默认 0.1，TL;DR 用 0.5。这是以后天天要调的超参。
- **π_{ref} 从哪来**：有 SFT 模型就直接用；没有（例如偏好数据是别的模型生成的）就先在 y_w 上做一轮 SFT 当 π_{ref}，否则数据和 π_{ref} 的分布对不上。训练全程冻结，只做前向。
- **数据必须成对、来自同一个 prompt**：推导要求，见第 4 节。
- **代码里最容易写错的地方**：`pi_logps` 是整条 response 的 log 概率，即 response 部分每个 token 的 log prob 之和，不含 prompt；shape 是 `(B,)`，每条序列一个标量。
- **读变体时看它改了哪一处**：几乎所有 DPO 变体都在三处动手——β；implicit reward 的形式（SimPO 去掉 π_{ref}，KTO 改成不需要成对，IPO 改 loss 形状）；偏好对怎么造。多模态方向的主战场是第三处，上面那批论文几乎没有一篇在改数学。
- **动手**：复现 IMDb（见第 6 节），扫 β，看奖励、KL、输出长度，以及 implicit reward 的 margin（y_w 的减去 y_l 的）分别怎么变。跑完一轮再回头看 3.5 节的梯度式子，会好读很多。

可以完全不看的：总结里的 3.2–3.4（论文第 4 节的推导部分）、4.1 的等价类与 Theorem 1 证明、4.2 的式 (10)、附录 A 全部、Plackett-Luce。

Q：DPO 还有哪些地方没看？

A：建议补两节：

- **6.4 GPT-4 判分能不能信**（总结 5.5、表 2）。摘要和对话的结论全靠 GPT-4 当裁判。作者找了 25 个人做人类评测来验证：人与 GPT-4 的一致率，大致等于人与人之间的一致率。另一个细节：换一种 prompt 写法，GPT-4 判出的 DPO win rate 会从 47% 变成 54%。以后做 VLM 后训练大概率也要用 LLM 当裁判，怎么证明裁判可信、prompt 会把结果带偏多少，这一节是现成的样板。
- **第 7 节的局限**（总结 §6）。作者自问 DPO 会不会也出现 reward 过度优化、图 3 右后期那个小幅下滑算不算——论文没给答案，后续一大批 DPO 变体正是冲着这个问题去的。

6.3 分布外泛化（表 1，CNN/DailyMail）可看可不看。

Q：GRPO 是 DPO 的升级版吗?

A：不是。GRPO 是 **PPO 的变体**，和 DPO 是两条不同的路线。

DPO 把 PPO 的整套循环都删了——RM、训练中的采样、value network、RL，一个不剩，换成离线数据上的一个损失。GRPO 只从 PPO 里删掉了 **value network**，其余全部保留：训练时照样现场采样，照样是 RL，照样有 clip 和 KL 约束。替代 value network 的办法是：对同一个 prompt 一次采一组 response，用这组的平均奖励当基线，高于平均的往上推，低于平均的往下压。

与 DPO 5.2 节的联系（这是我的理解，不是两篇论文的原话）：两者都在解决「PPO 需要一个难估的基线来降方差」这个痛点，方向相反——DPO 从数学上让它消失，GRPO 在同一个 prompt 上多采几条，用平均分把它直接估出来。

所需数据也不同：DPO 要成对的偏好数据；GRPO 要给每条 response 打一个分，最常见的是规则判对错（数学答案对不对、代码能不能跑通），这也是它在 reasoning 方向流行起来的原因。

阅读提示：GRPO 没有单独的论文，它是 DeepSeekMath（arXiv:2402.03300）里的一节，那篇大部分篇幅在讲数学数据与预训练。之前 Spurious Advantage 那篇的总结第 2 节已经讲过 GRPO 的 advantage 怎么算，可以对照着读。

参考来源（论文之外的事实）

- 官方代码仓库 README（硬件、时长、命令）：https://github.com/eric-mitchell/direct-preference-optimization
- PEA-DPO：https://www.alphaxiv.org/abs/2608.19598
- P²-DPO：https://www.alphaxiv.org/abs/2606.03376
- Context Blindness in DPO：https://www.alphaxiv.org/abs/2608.12158
- Groc-PO：https://www.alphaxiv.org/abs/2607.13712
- ADAPT：https://www.alphaxiv.org/abs/2606.31054
- MCPO：https://www.alphaxiv.org/abs/2609.04947
- Graded Preference Optimization：https://www.alphaxiv.org/abs/2609.36628
- Inducing Emergent Misalignment from Reward Hacks with Iterative DPO：https://www.alphaxiv.org/abs/2609.06649
- Context-Grounding Gains…: Auditing GRPO, SFT, and DPO：https://www.alphaxiv.org/abs/2609.00925
- It Takes Two: Your GRPO Is Secretly DPO：https://arxiv.org/html/2510.00977v1